

1. hpfp int_converter(int input)

int를 입력값으로 받고 hpfp 로 변환하기 위해 s, exp, frac, E 등등 hpfp의 변수들을 unsigned int로 설정했습니다. s, exp, frac 등을 각각 구하기 위해 input 값을 넣을 temp 변수를 만들었습니다. 처음에는 input의 부호에 따라 s의 값을 구했습니다. temp에 input의 절댓값을 넣어준 뒤에 temp를 오른쪽으로 shift 해가며 E의 값을 구하고 exp의 값도 구했습니다. 이때 exp 가 30을 넘어가면 지수부분이 11111이 돼서 무한대가 되기 때문에 s 가 0일때는 +무한대, s가 1일때는 -무한대가 되도록 설정했습니다. temp에 다시 input의 절댓값을 넣어준 뒤 10비트의 frac을 추출하기 위해 temp의 길이를 구하고 temp의 길이가 10비트를 넘어갈때는 상위 비트만 유지시키고, 10비트이하일때는 temp를 왼쪽으로 shift해서 나머지 비트를 0으로 채웠습니다. s, exp, frac을 모두 구한뒤 total 변수에 합쳐서 hpfp를 구했습니다.

2. int hpfp_to_int_converter(hpfp input)

hpfp를 입력값으로 받고 int로 변환하기 위해 s, exp, frac 값을 shift와 &연산자를 이용해 input 으로부터 분리해서 변수로 저장했습니다. 만약 지수부분이 모두 1인 경우 조건에 따라 TMAX 와 TMIN을 반환하도록 설정했습니다. 변환하는 값이 정수이기 때문에 E가 0보다 작은 경우에는 0을 반환하도록 설정했고, E보다 큰경우 total 값이 $(2^E) * (1 + \text{frac} / (2^{10}))$ 이 되도록 만들었습니다. 이때 $\text{frac} / 1024$ 가 소수점까지 정확한 값을 표시하도록 하기 위해 (float)을 앞에 붙였습니다. 마지막에는 s값에 따라 부호를 설정해서 정확한 int값을 반환할 수 있도록 만들었습니다.

3. hpfp float_converter(float input)

float를 입력값으로 받고 hpfp 로 변환하기 위한 함수를 구현했습니다. 기본적인 것은 1번 함수하고 비슷하지만, float은 입력 값이 1보다 작은 경우도 있기 때문에 그런 경우에 2를 곱해서 temp를 1과 2사이로 맞추고, E를 감소시켰습니다. 또 exp가 0보다 작은 경우에는 부호만 살려서 hpfp의 exp와 frac이 모두 0이 되도록 했습니다. 그리고 float 은 int와 달리 소수점 형식이기 때문에 정규화된 temp에다가 1을 빼준뒤(암묵적인 frac의 맨 앞 비트 1) 2^{10} 을 곱해서 frac 부분을 얻었습니다.

4. hpfp_to_float_converter(hpfp input)

hpfp를 입력값으로 받고 float로 변환하기 위한 함수를 구현했습니다. 2번 함수와 비슷한 구조지만, float는 int값처럼 tmin, tmax 가 없기 때문에 +무한대, -무한대를 1.0/0.0, -1.0/0.0 으로 표현했습니다. 또 NaN값은 부정형 0.0/0.0으로 표현했습니다. denormalized numbers인 exp가 0일때는 frac에 1을 안더하고, 그 외에 normalized numbers인 경우에는 $\text{frac} / 1024$ 에 1을 더했습니다. 그리고 float는 int처럼 shift를 사용할 수 없기 때문에 result를 구하기 위해 total을 E만큼 곱하거나 나누도록 설계했습니다. 마지막에는 s값에 따라 부호를 설정해서 최종 float값을 구했습니다.

5. addition_function(hfpf a, hfpf b)

hfpf끼리 더하는 함수를 만들기 위해서 각 hfpf의 s, exp, frac부분을 변수로 설정했습니다. 첫 부분에는 예외 상황부터 설정했습니다. 두 hfpf의 값이 모두 무한대인데 부호가 다를 경우에는 NaN을, 그 외(부호가 같은 무한대, 한쪽이 무한대인데 다른쪽은 Normal Value)에는 무한대를 반환하도록 했습니다. 그리고 어느 한쪽이라도 NaN인 경우에는 NaN을 반환하도록 했습니다. 덧셈을 시작하기 전에 normalized numbers는 frac부분에 암묵적인 1이 있기 때문에 $\text{frac} + 1024$ 을 해주고, denormalized numbers는 1024를 안더하고 그대로 사용하도록 했습니다. 두 hfpf 값 중 exp가 큰값의 s1, exp1를 s, exp로 설정하고, 두 exp의 차이를 diff로 둔 뒤에 두 hfpf의 frac 부분을 맞추기 위해 작은 hfpf의 frac을 diff 만큼 shift 했습니다. 그 뒤 부호가 같은 경우에는 두 frac을 더하고, 다른 경우에는 더 큰 frac에서 작은 frac을 빼고 큰 frac의 hfpf 부호로 s를 설정했습니다. 그 뒤 frac이 2.0 이상이거나 1.0미만이 되지 않도록 조정했습니다. 마지막으로 exp가 31이상인 경우에는 무한대를, 0보다 작은 경우에는 0을 반환하도록 오버플로우와 언더플로우를 정했습니다.

6. multiply_function(hfpf a, hfpf b)

hfpf끼리 곱하는 함수에서는 한 hfpf가 무한대일때 다른 hfpf가 같은 부호면 + 무한대를, 반대 부호면 - 무한대를 반환하고, 무한대 * 0 이면 NaN을 반환하는 등 예외적인 상황부터 우선으로 처리했습니다. frac변수에는 두 hfpf의 frac 부분을 곱한뒤 상위 10비트만 남기고, exp는 $(\text{exp1}-15)+(\text{exp2}-15)+15$ 로 계산했습니다. temp, temp2, near, remain은 round-to-even을 처리하기 위해 쓰였습니다. near에는 반올림할 대상의 가장 가까운 비트를, temp2에는 그 뒤의 비트들을 저장했습니다. 그리고 만약 temp2 가 0이면 remain을 0으로, 적어도 하나의 비트라도 1이면 remain을 1로 설정하고, near가 1이고 remain 이 1이면 올림을 수행하고 near가 1이고 remain 이 0이면 frac이 홀수일 때 1을 더해서 짝수가 되도록 했습니다. 그 뒤에는 5번 함수와 마찬가지로 frac의 범위를 조정하고 result값을 반환하는걸로 마무리했습니다.

7. *comparison_function(hfpf a, hfpf b)

두 hfpf끼리 비교하는 함수를 만들기 위해 각 hfpf의 s, exp, frac 을 변수로 저장했습니다. 먼저 특수 경우인 둘중 하나라도 NaN일때, 두 hfpf가 무한대일 때, 한쪽은 무한대고 한쪽은 무한대가 아닐 때를 먼저 만들었습니다. 그 다음에 부호에 따라 hfpf를 결정하고, 부호가 같을시에는 exp에 따라서, exp도 같을시에는 frac에 따라서 부호를 정하도록 만들었습니다.

8. *hfpf_to_bits_converter(hfpf result)

hfpf를 bits단위로 표현하기 위한 함수를 구현하기 위해 16비트 길이를 저장할 수 있는 문자열을 만들었습니다. hfpf의 result 값을 bits의 0인덱스부터 시작해서 넣었는데, 문자열에 숫자 비트를 표현하기 위해 '0'을 더해서 ASCII코드에서 0과 1을 나타낼 수 있게 했습니다.